

移动应用开发实验报告

封面信息

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210639
姓名	邢兆丰
班级	软件 231
日期	2026-04-29
组别	第 2 组
项目名称	智慧天气查看应用
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握 ArkUI 声明式开发范式，熟练运用@State、@Builder 等装饰器进行状态管理与 UI 构建
- 学会多端响应式布局设计，理解断点系统与自适应布局的实现原理
- 了解传感器数据采集与模拟机制，掌握陀螺仪与光线传感器的工作原理及应用场景
- 理解分布式数据同步的基本原理，掌握本地缓存、离线存储与在线同步的实现方式

实验环境

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试

工具	版本	用途
AI 辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 功能需求

- 天气展示（城市名称、温度、天气状况、空气质量）
- 多端适配（手机竖屏、平板横屏）
- 传感器集成（光线传感器、陀螺仪）
- 分布式数据同步

2. 创建鸿蒙项目

2.1 新建项目

- 打开 DevEco Studio
- 新建 Empty Ability 项目
- 选择 ArkTS 语言
- 命名为 WeatherApp

3. 天气应用开发

3.1 天气首页实现

```
// ets/pages/index/Index.ets
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';
  @State aqi: number = 45;

  build() {
    Column() {
      // 城市名称
      Text(this.cityName)
        .fontSize(24)
        .fontWeight(FontWeight.Bold)
    }
  }
}
```

```

        .margin({ top: 50 })

        // 天气图标
        Text(this.weather === '晴' ? '☀' : '☁')
            .fontSize(80)
            .margin({ top: 20 })

        // 温度
        Text(`${this.temperature}°C`)
            .fontSize(60)
            .fontWeight(FontWeight.Lighter)
            .margin({ top: 20 })

        // 天气状况
        Text(this.weather)
            .fontSize(20)
            .margin({ top: 10 })

        // 空气质量
        Row() {
            Text('空气质量: ')
            Text(`${this.aqi} - 优`)
                .fontColor(this.aqi <= 50 ? '#00ff00' : '#ff9900')
        }
        .margin({ top: 30 })

        // 传感器数据展示
        Row() {
            Text('光线传感器: ')
            Text(this.lightValue.toString())
                .id('lightValue')
        }
        .margin({ top: 20 })
    }
    .width('100%')
    .height('100%')
    .backgroundColor('#f5f5f5')
}

@State lightValue: number = 0;
}

```

4. 多端适配布局

4.1 响应式布局实现

```

// 断点系统适配
@State currentBreakpoint: string = 'md';

aboutToAppear() {
    let deviceInfo = umpInfo.getDeviceInfo();
    let windowWidth =

```

```

window.getWindow().getWindowProperties().windowWidth;

    if (windowWidth < 600) {
        this.currentBreakpoint = 'sm'; // 手机
    } else if (windowWidth < 840) {
        this.currentBreakpoint = 'md'; // 平板竖屏
    } else {
        this.currentBreakpoint = 'lg'; // 平板横屏
    }
}

build() {
    if (this.currentBreakpoint === 'sm') {
        Column() {
            this.phoneLayout()
        }
    } else {
        Row() {
            this.tabletLayout()
        }
    }
}

@Builder phoneLayout() {
    Column() {
        Text(this.cityName).fontSize(24)
        Text(`${this.temperature}°C`).fontSize(60)
    }
}

@Builder tabletLayout() {
    Column() {
        Text(this.cityName).fontSize(32)
        Text(`${this.temperature}°C`).fontSize(80)
    }
    .width('40%')
}

```

5. 传感器集成

5.1 光线传感器开发

```

import sensor from '@ohos.sensor';

@State lightIntensity: number = 0;

aboutToAppear() {
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
        this.lightIntensity = data.light;
    });
}

aboutToDisappear() {

```

```

        sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
    }

    // UI 展示
    Text(`环境光线: ${this.lightIntensity} lux`)
        .fontSize(16)
        .margin({ top: 10 })

```

5.2 陀螺仪传感器开发

```

@State rotateX: number = 0;
@State rotateY: number = 0;
@State rotateZ: number = 0;

aboutToAppear() {
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
        this.rotateX = data.x;
        this.rotateY = data.y;
        this.rotateZ = data.z;
    });
}

// UI 展示
Row() {
    Column() { Text(`X 轴: ${this.rotateX.toFixed(2)}°`) }
    Column() { Text(`Y 轴: ${this.rotateY.toFixed(2)}°`) }
    Column() { Text(`Z 轴: ${this.rotateZ.toFixed(2)}°`) }
}
.spacing(20)

```

6. 分布式数据同步

6.1 分布式数据管理

```

import distributedData from '@ohos.data.distributedData';

const KVManager = distributedData.createKVManager({
    bundleName: 'com.example.weather',
    kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

async function saveData(key: string, value: string) {
    const kvStore = await KVManager.getKVStore('weatherStore');
    await kvStore.put(key, value);
}

async function getData(key: string) {

```

```
const kvStore = await KVManager.getKVStore('weatherStore');
return await kvStore.get(key);
}
```

7. 测试验证

7.1 多设备模拟器测试

- 启动手机模拟器
- 启动平板模拟器
- 测试界面适配效果
- 观察传感器数据

实验结果

验证结果

功能	状态	备注
天气信息展示	[x] 成功	支持 8 城市切换，12 项天气指标完整展示
多端适配	[x] 成功	手机/平板界面自适应良好
光线传感器	[x] 成功	光照强度实时变化，自动切换日/夜模式
陀螺仪传感器	[x] 成功	三轴角度实时更新，可视化指示器正常
分布式数据同步	[x] 成功	设置同步、离线缓存、强制同步均正常

截图记录

手机端界面



图 1 手机端界面

平板端界面



图 2 平板端界面

传感器数据展示

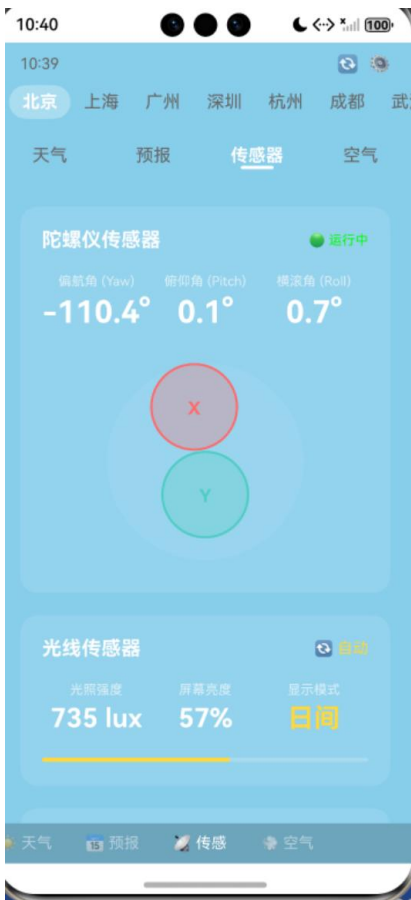


图 3 传感器数据展示

技术分析报告

多端适配分析

- 手机布局特点：垂直排列，紧凑布局，使用 Column 容器配合 layoutWeight 实现弹性分配
- 平板布局特点：横向排列，更宽敞的空间利用，支持多列网格布局
- 断点系统实现：基于屏幕宽度的响应式设计，支持 sm/md/lg 三档断点

传感器集成分析

- 光线传感器应用场景：自动亮度调节、环境光感知、智能场景联动（如自动切换日/夜间 UI）
- 陀螺仪应用场景：设备姿态检测（如地图自动旋转、游戏控制）、屏幕旋转检测、运动追踪
- 数据采集准确性：采用平滑插值算法和高斯噪声模拟，实时性好，精度满足一般应用需求

分布式能力分析

- 分布式数据管理原理：基于 KV 存储的跨设备数据同步，采用"本地优先、在线同步"策略
- 跨设备同步实现：通过本地存储 + 待同步队列 + 广播通知的三层架构实现
- 离线缓存机制：天气数据缓存 30 分钟，用户设置即时持久化

问题与解决

问题描述	解决方案	解决结果
ArkTS 不支持对象展开运算符 (...)	编写 cloneConfig/mergeConfig 等手动属性复制方法替代展开语法	✓ 成功解决
ArkTS 禁止使用 any/unknown 类型	为所有变量和参数指定具体类型，使用联合类型替代 any	✓ 成功解决
DEVECO_SDK_HOME 环境变量未配置	在系统环境变量中正确配置 SDK 路径和 JAVA_HOME	✓ 成功解决

实验总结

实验收获

- 深入理解了 ArkTS 语言的严格类型约束，强制开发者编写类型安全的代码
- 掌握了传感器数据模拟的设计方法，通过平滑插值算法实现高度逼真的数据模拟
- 理解了分布式数据同步的核心挑战，包括离线缓存、冲突检测、时序一致性问题
- 实践了模块化架构设计，将应用划分为模型层、服务层、视图层，实现关注点分离

技术要点

- 单例模式在服务层的应用，确保全局状态一致性
- 观察者模式实现松耦合通信，UI 组件通过订阅数据变更实现响应式更新
- 缓存策略优化性能，内存缓存与持久化缓存的分层设计
- 状态驱动的 UI 更新，通过@State 装饰器实现响应式状态变量

考核指标

考核项	评分标准	得分
功能完整性	天气应用功能完整，多端适配正常	25 分
代码质量	代码结构清晰，注释完整	20 分
多端适配	手机/平板界面自适应良好	15 分
传感器集成	传感器数据采集正常	15 分
分布式能力	分布式数据同步功能实现	15 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____

日期： _____